MEDIBOT

TECHNICAL GUIDE · OCTOBER 2026

MediBot
Stack & Architecture

Implementation guide to the current safety-first medicine information prototype: runtime, request flow, data model, integrations, setup, tests, and known gaps.

Bun · TypeScript · SQLite · Groq · Tavily

01 · SCOPE AND STATUS

What this document describes

This guide describes the code currently in the MediBot project—not a production deployment and not every feature proposed in the original architecture PDF.

IMPLEMENTED

- Bun HTTP server and static browser UI
- TypeScript tool-calling agent using Groq
- SQLite catalogue and safety-term tables
- CSV importer for the downloaded 1mg Kaggle file
- Optional Tavily web search for selected medical-reference domains
- Input, tool, and generated-answer checks
- Disclaimer acknowledgement and in-memory chat context

NOT YET IMPLEMENTED / NOT VERIFIED

- Complete official restricted-drug data
- MID workbook ingestion or composition normalization
- Live pharmacy APIs, permitted scraping, or real-time stock
- Nearby places or location-based search
- Production authentication, deployment, monitoring, or audit trail
- Clinical-grade accuracy validation

> **Intended use:** prototype and educational demonstration only. It is not a medical device, pharmacist, or substitute for professional advice. Do not use it to make medication decisions.

The initial project architecture PDF defines the desired problem and guard flow. This guide records implementation status as of October 2026; scope gaps are called out instead of presented as completed features.

02 · TECHNOLOGY STACK

Stack and responsibilities

Technology	Role	Project usage
Bun	JavaScript runtime and project tooling	Runs the server, SQLite bindings, development watcher, tests, and scripts.
TypeScript	Application language and static checks	Server routes, data access, guards, import code, integrations, and agent orchestration.
bun:sqlite	Local relational persistence	Medicine/product data, prices, safety terms, health-info rows, and disclaimer session records.
Vanilla HTML/CSS/JS	Browser interface	Chat, disclaimer acknowledgement, example prompts, Markdown-like rendering, and answer citations.
Groq API	LLM inference and tool selection	Configured model default: `openai/gpt-oss-120b`; model IDs/access may change by account.
Tavily Search API	Optional web search	General-information fallback; only when `TAVILY_API_KEY` is set.
Bun test + TypeScript compiler	Verification	`bun test` exercises guards/CSV/search parsing; `bun run typecheck` checks TypeScript.

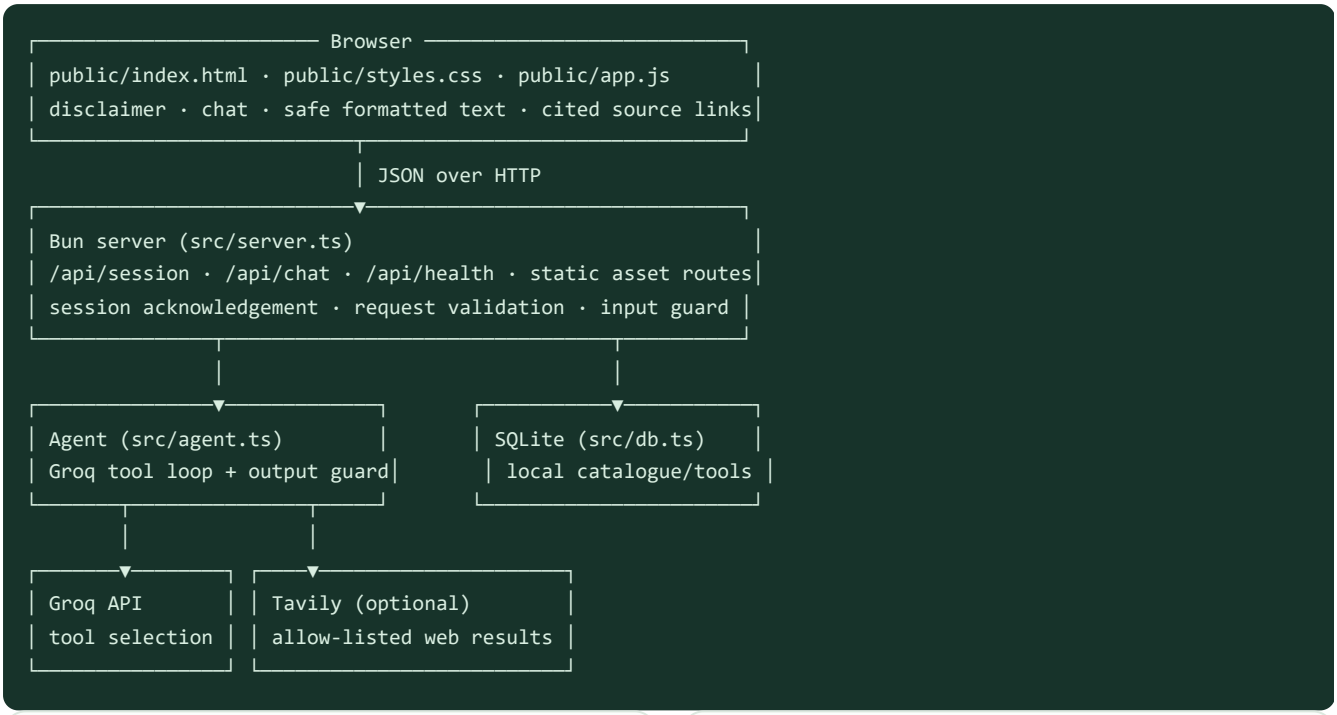
Why the stack fits a prototype

Bun provides a compact TypeScript runtime with a built-in SQLite API, avoiding a separate database server for a local demo. Plain browser assets keep the interface dependency-light. External models and search are called server-side so provider keys do not go to the browser.

> **Configuration:** local secrets belong in the ignored `.env` file. The checked-in `.env.example` documents names only. Never commit API keys or include them in screenshots/PDFs.

03 · COMPONENT ARCHITECTURE

System components



Browser

Displays responses as safe DOM text nodes and formats a limited Markdown subset. Source URLs are accepted only as HTTPS links.

Bun server

Checks request size/shape, records disclaimer acknowledgement, applies Guard 1, coordinates the agent, and returns JSON responses.

Agent

Calls Groq with local tool schemas; executes bounded tool steps; optionally runs one web search; checks final generated text.

SQLite

Local persistent records for catalogue/source entries and sessions. Conversational messages themselves stay in a process-memory map.

04 · REQUEST LIFECYCLE

Chat request, guard by guard

1. **Session:** browser sends `POST /api/session` after the user acknowledges the disclaimer. Server inserts an acknowledgement timestamp and generates a random session ID.
2. **Validate:** `POST /api/chat` checks that the session exists and the message contains 1–2,000 characters.
3. **Guard 1:** the server checks restricted-term fuzzy matching and a limited indirect-phrase rule on every message. On a match, it returns the required hardcoded line before calling Groq.
4. **Context:** last messages are kept in server memory (bounded to six user/assistant turns). This map resets when the server process restarts.
5. **Agent:** Groq receives the system instructions and local tools. If Tavily is configured, the medical-search tool is also available.
6. **Tool guards:** each catalogue tool checks its medicine input before querying data. Web-search queries are checked before the Tavily request.
7. **Guard 2:** generated response text is checked for known restricted names and a small set of diagnosis phrases; the agent may request up to two rewrites.
8. **Render:** response text, source list, and internet-information notice are returned to the browser; citation links are shown separately.

> **Important boundary:** these checks cannot guarantee detection of every restricted product, brand, misspelling, or diagnosis phrase. The bundled restricted list is incomplete demo data, and the output-language matcher is not a clinical classifier.

Relevant routes

Route	Purpose
`GET /api/health`	Reports server status and whether Groq/Tavily keys are configured; never returns keys.
`POST /api/session`	Creates a session after explicit disclaimer acknowledgement.
`POST /api/chat`	Validates, guards, runs the agent, and returns answer plus citations.
`GET /`	Serves the browser app.

05 · SQLITE MODEL

Database tables and relationships

```
medicines 1 ── * prices
medicines 1 ── 0..1 health_info
restricted_terms (independent safety vocabulary)
sessions (disclaimer acknowledgement metadata)
```

Table	Main fields	Meaning / limitation
`medicines`	id, brand, manufacturer, composition, normalized_composition, mrp, restricted, pack_size, rating, data_source, source_price, source_mrp	Product row. Imported CSV lacks ingredient/manufacturer; these fields are marked unprovided, not inferred.
`prices`	medicine_id, pharmacy, price, eta_days, in_stock, source_type	Source price snapshot or earlier simulated demo values. Imported rows have unknown stock and ETA.
`health_info`	medicine_id, side_effects, interactions	Only old demo seed records; these are cleared when the importer replaces the catalogue.
`restricted_terms`	term, ingredient	Currently eight demonstration aliases/ingredient terms, not an official list.
`sessions`	id, disclaimer_acknowledged_at, created_at	Persists session acknowledgement metadata. Conversation text is not stored here.

Foreign keys are enabled. Deleting medicines cascades to dependent prices and health-info rows. The 1mg importer uses a transaction when replacing medicine rows, preserving the independent sessions and restricted-term table.

> **Privacy note:** chats are stored only in process memory; however session IDs and acknowledgement timestamps persist in SQLite. There is no production account system, deletion workflow, or multi-process session store.

06 · DATA PIPELINE

Kaggle CSV: actual import and caveats

```
data/raw/1mgData.csv
    │  CSV parser · required-header check · numeric parsing
    │  skip blank names · deduplicate exact row fingerprints
    ▼
data/medibot.sqlite
    ├── medicines (source columns + unavailable fields left explicit)
    └── prices (Kaggle snapshot source label; unknown stock/ETA)
```

CATALOG

15,948

Products imported after exact duplicate removal.

PRICES

11,648

Parseable source price entries imported.

MISSING

4,325

Raw CSV rows with an unparseable price; count is before deduplication.

CSV header	Stored as	Notes
`Name`	`medicines.brand`	Product title/name, not necessarily a normalized brand.
`pack_size`	`medicines.pack_size`	Source text retained.
`rating_count`, `rating`	Catalogue metadata	Rating is parsed only if between 0 and 5.
`price`	`medicines.source_price` and price row	Source semantics not independently verified; not a current quote.
`mrp`	`medicines.source_mrp`	Preserved as supplied. CSV column labels are not reinterpreted.

> **Observed anomaly:** the imported source price is greater than source MRP in many rows (11,648 in the import report). Verify the dataset documentation and fields before calculating savings or drawing price conclusions.

The Kaggle license/source URL and reliable capture date still need to be documented. The MID Excel workbook and therapeutic-class workbook are present under `data/raw/` but have not been inspected/imported.

07 · AGENT TOOLS

What the model can call

Tool	Data boundary	Important behavior
`get_composition`	SQLite medicine record	Imported 1mg rows report composition unavailable; model must not fill it from memory.
`find_alternatives`	Exact normalized composition and strength	Returns no suggestions for entries marked unverified. Kaggle file has no composition field.
`compare_prices`	SQLite price records	Distinguishes Kaggle snapshots from simulated demo values; no live pharmacy feed exists.
`get_drug_info`	SQLite health_info	Kaggle data does not provide this information; old demo health rows are removed by catalogue import.
`search_medical_web` (optional)	Tavily excerpts from allow-listed domains	One search per agent call; produces HTTPS citations. Not used to decide restriction status or prescribe.

Web source policy implemented

Search requests use Tavily with domains including MedlinePlus, NHS, DailyMed, FDA, CDSCO, WHO, and NCCIH. Results require HTTPS, duplicate links are skipped, and excerpts containing a known restricted term are discarded. The agent prompt labels excerpts untrusted and tells the model not to follow instructions in them.

> **Privacy:** the generated general search query is sent to Tavily. A small pattern blocks some personal-looking queries, but this is not a complete privacy detector. Users are told not to include personal identifiers or private health details.

Citations: source title and URL are sent separately to the browser; the UI validates HTTPS before making external links and opens them with `noopener noreferrer`. If no excerpts are returned, the agent is instructed not to answer from memory.

Safety is layered—and still incomplete

Layer	Current mechanism	Residual risk
Input	Known terms, simple edit-distance typo matching, selected indirect-reference patterns	Small demo vocabulary; unseen brand names, languages, phrasing, ingredient aliases, or misspellings may pass.
Tool input	Catalogue tools re-run restricted-term checks	Same incomplete list; only protects paths that match the current checks.
Output	Restricted-name scan and regex for some diagnosis phrases; up to two rewrites	Language rules do not understand all clinical claims or context and cannot guarantee zero unsafe outputs.
Alternatives	Database query requires identical normalized composition/strength	Imported Kaggle rows have no composition and therefore cannot be compared.
Sources	Trusted-domain search and visible citations	Domain allow-list does not mean every page is accurate or relevant; users should verify source and date.

Credential and data-handling points

- Groq and Tavily keys are server-side environment variables and should remain only in local `.env`, which is gitignored.
- Search queries are sent to an external provider; avoid submitting personal or sensitive information.
- There is no user login, rate limit, CSRF defense, production TLS configuration, or deployment hardening in this local prototype.
- Do not expose the dev server to the public internet or treat it as a clinical service.

> **Before real use:** replace the small seed vocabulary with an authoritative and reviewed restricted list, develop a representative safety test set, measure false blocks and misses, and obtain qualified compliance/clinical review.

09 · LOCAL SETUP AND OPERATIONS

Run, configure, import, verify

1. 1 Install Bun, open the project folder, and run `bun install`.
2. 2 Copy `.env.example` to `.env`; set `GROQ_API_KEY`. Default model: `openai/gpt-oss-120b`.
3. 3 Optionally set `TAVILY_API_KEY` for cited web fallback.
4. 4 Start with `bun run dev`; open `http://localhost:3000`. If port 3000 is occupied, stop the existing server before starting another.
5. 5 Check `GET /api/health` for `groqConfigured` and `webSearchConfigured`. These are booleans; API keys are not returned.
6. 6 Run `bun run import:1mg:check` to preview the CSV, then `bun run import:1mg` to replace catalogue products.
7. 7 Verify with `bun run typecheck` and `bun test`.

> **Importer effect:** importing replaces medicine rows and cascades deletion of dependent prices and health-info rows. It retains sessions and restricted terms. Back up `data/medibot.sqlite` before future reimports if its records matter.

Configuration keys

Variable	Purpose	Default
`GROQ_API_KEY`	Groq inference credential	Unset (demo fallback path)
`GROQ_MODEL`	Groq model identifier	`openai/gpt-oss-120b`
`TAVILY_API_KEY`	Optional web-search credential	Unset (no online search)
`PORT`	HTTP listening port	`3000`
`DB_PATH`	SQLite file path	`./data/medibot.sqlite`

10 · ROADMAP

Recommended path forward

DATA QUALITY

Establish provenance

Record dataset owner/source URL, version or capture date, license, attribution, and transformation rules. Resolve the source price/MRP anomaly before savings calculations.

SAFETY

Authoritative restrictions

Obtain official current restricted-drug sources; normalize ingredient and brand aliases; get qualified review; expand regression and typo tests.

COMPOSITION

Validated matching

Inspect MID workbook schema and license. Normalize ingredients, strengths, salts, dosage form, and pack size. Keep confidence/provenance per match.

PRICING

Approved current feed

Use a partner API or explicitly permitted source. Store timestamp, product identifier, pack, availability, and provenance for every quote.

OPERATIONS

Harden deployment

Add authentication if needed, rate limits, request timeouts, secure hosting/TLS, secret management, monitoring, and privacy/deletion policies.

EVALUATION

Measure before claims

Evaluate catalogue matching, citation quality, restriction recall/false blocks, diagnosis-language escapes, latency, and user feedback.

> **Scope reminder:** the current application uses web search for general reference fallback, not pharmacy scraping. Add no scraper until the target source's terms and permission explicitly allow it.

Project references: included MediBot Architecture and Data Sources PDF; current project source in `src/`, browser UI in `public/`, importer in `src/import-1mg.ts`, and source data under `data/raw/`. Dataset license and exact Kaggle attribution were not confirmed during documentation preparation.